

AI & Automation Domain — Task 2

System Report: End-to-End RAG Knowledge Management Pipeline

Stack	LangChain · Google Gemini 1.5 Flash · ChromaDB · FastAPI
Embedding Model	gemini-embedding-001 (768-dim, free tier)
Chat Model	gemini-1.5-flash (free developer tier)
Vector Store	ChromaDB — local persistent HNSW index
Reference	pixegami LangChain RAG Tutorial (youtube.com/watch?v=tcqEUSNCn8I)
Deadline	21 June 2026

1. Executive Summary

This report documents the design, implementation, and evaluation of a five-stage Retrieval Augmented Generation (RAG) pipeline built for the E-Cell NIT Trichy AI & Automation Task 2. The system ingests raw technical PDF documents, segments and indexes them into a persistent vector database, and serves semantically grounded answers through a production-ready FastAPI endpoint — following the approach demonstrated in the pixegami LangChain RAG tutorial.

All five required stages are implemented in a single modular Python file (pipeline.py), with each stage runnable independently or as part of a full end-to-end pipeline.

2. System Architecture

Stage	Component	Technology
Stage 1	Document Ingestion & Text Segmentation	LangChain DirectoryLoader + PyPDFLoader
Stage 2	Embedding Generation & Indexing	Gemini gemini-embedding-001 + ChromaDB
Stage 3	LLM Inference & Context Orchestration	Gemini 1.5 Flash + ChatPromptTemplate
Stage 4	Pipeline Evaluation	Custom CR / F / AR / L / QR metrics
Stage 5	API Deployment	FastAPI + uvicorn, POST /query endpoint

3. Stage 1: Document Ingestion & Text Segmentation

3.1 Document Loading

Documents are loaded using LangChain's DirectoryLoader with PyPDFLoader as the backend, scanning the data/ directory recursively for all .pdf files. The corpus used consists of 5 landmark AI and security documents totalling 150 raw pages.

Document	Pages	Type
Attention Is All You Need (Vaswani et al., 2017)	15	Technical Document
BERT: Pre-training of Deep Bidirectional Transformers	16	Technical Document
RAG for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)	13	Technical Document
GPT-4 Technical Report (OpenAI, 2023)	100	Technical Document
NIST SP 800-53 Rev 5 — Security & Privacy Controls	6	Compliance Regulation

3.2 Text Cleaning

Raw PDF text contains layout noise from the extraction process. A custom clean_text() function removes this before chunking:

Noise Type	Cleaning Method
Hyphenated line breaks	re.sub(r' -\n', '', text)
Standalone page numbers	Regex: ^\s*(\d+ Page \d+ of \d+)\s*\$

Confidentiality banners	Regex on common boilerplate phrases
Excessive whitespace	Collapse 3+ newlines to 2, 2+ spaces to 1

3.3 Text Chunking Strategy

Text is split using LangChain's RecursiveCharacterTextSplitter with the following configuration:

Parameter	Value	Justification
chunk_size	1000 characters	Captures enough context for a full technical explanation; suitable for dense PDF content
chunk_overlap	500 characters (50%)	Ensures information spanning page boundaries is captured by at least one chunk
length_function	len()	Character-based counting; consistent across all document types
add_start_index	True	Stores character offset in metadata for provenance tracking

Result: 842 chunks produced from 150 pages across 5 documents.

3.4 Document Type Classification

Each chunk is automatically classified into one of four task-specified document types using keyword and regex heuristics stored in chunk metadata:

Document Type	Example Keywords
Standard Operational Procedure	SOP, step-by-step, work instruction, operating procedure
Corporate Policy Framework	policy, governance, framework, corporate policy
Compliance Regulation	compliance, GDPR, ISO, audit, regulatory, legal requirement
Technical Troubleshooting Log	error, bug, troubleshoot, incident, root cause, resolution
Technical Document	Fallback for unmatched chunks

4. Stage 2: Embedding Generation & Indexing

4.1 Embedding Model Selection

Option	Model	Dimensions	Cost	Selected
Gemini	gemini-embedding-001	768	Free (1000 req/day)	YES
OpenAI	text-embedding-ada-002	1536	Paid per token	No
Local	all-MiniLM-L6-v2	384	Free, local GPU	No

Gemini embeddings were selected as they are free, require no local GPU, and produce high-quality 768-dimensional semantic representations suitable for technical document retrieval.

4.2 Vector Store: ChromaDB

ChromaDB is used as the persistent local vector database, matching the approach in the reference tutorial. Key configuration:

Parameter	Value
Similarity metric	Cosine similarity
Storage location	models/chroma/ (local disk)
Index type	HNSW (Hierarchical Navigable Small World) — ChromaDB default
Persistence	db.persist() called after indexing

4.3 Rate Limit Handling

The Gemini free tier allows 100 embedding requests per minute and 1000 per day. Embedding is performed in batches of 10 chunks with a 15-second pause between batches to respect the rate limit. This allows the full 842-chunk corpus to be embedded over multiple sessions.

5. Stage 3: LLM Inference & Context Orchestration

5.1 Retrieval

For each query, the system embeds the query using the same Gemini embedding model, performs cosine similarity search in ChromaDB with k=3, and returns the top-3 most relevant chunks with their similarity scores. This matches the reference tutorial's `similarity_search_with_relevance_scores(query, k=3)` approach.

5.2 Anti-Hallucination Guardrails

Three layers of protection are implemented to ensure contextual groundedness:

Layer	Mechanism	Description
Guardrail 1	Similarity Floor (0.3)	If top chunk scores below 0.3, return 'I don't know' without calling the LLM
Guardrail 2	Strict System Prompt	LLM explicitly forbidden from using knowledge outside the retrieved context
Guardrail 3	Source Citation Requirement	Every answer must cite source filename and page — hallucinations are immediately de

5.3 LLM Configuration

Parameter	Value	Justification
Model	gemini-1.5-flash	State-of-the-art, 1M token context window, free tier
Temperature	0.1	Low temp maximises faithfulness to context, reduces hallucination
Max output	Default	Sufficient for concise grounded answers

5.4 Prompt Template

The strict prompt template (adapted from the reference tutorial):

```
Answer the question based ONLY on the following context. Do NOT use any outside knowledge.
If the answer is not in the context, say exactly: 'I don't have enough information in the
provided documents to answer this.' Always cite the source document and page number
inline. [Context: ...] Question: {question}
```

6. Stage 4: Pipeline Evaluation

6.1 Metrics Definition

Metric	Full Name	Definition
CR	Context Relevance	Fraction of expected topic keywords found in the top retrieved chunks
F	Faithfulness (Groundedness)	Fraction of answer sentences grounded in retrieved context words
AR	Answer Relevance	Fraction of query keywords that appear in the final answer
L	Inference Latency	Wall-clock time from query receipt to response (seconds)
QR	Query Resolution Rate	Fraction of queries where top chunk scored above similarity floor (0.3)

6.2 Evaluation Queries

#	Query	Expected Topics
1	What is the attention mechanism in transformers?	attention, query, key, value, softmax
2	How does BERT use masked language modelling?	mask, token, pretrain, bert
3	What is retrieval augmented generation?	retrieval, generation, document, knowledge
4	What are the contributions of the GPT-4 technical report?	capabilities, evaluation, multimodal
5	What security controls does NIST recommend?	access, control, authentication, security

6.3 Expected Performance

Metric	Expected Range	Target
CR — Context Relevance	0.60 — 0.90	> 0.70
F — Faithfulness	0.70 — 0.95	> 0.80
AR — Answer Relevance	0.50 — 0.85	> 0.65
L — Latency (s)	2.0 — 6.0	< 5.0
QR — Resolution Rate	0.80 — 1.00	1.00

7. Stage 5: API Deployment

7.1 Endpoints

Method	Path	Description
GET	/	Health check — confirms server is running
GET	/health	System status, model names, and chroma readiness
POST	/query	Submit a question, receive grounded answer + sources
GET	/eval	Return latest Stage 4 evaluation results
GET	/docs	Auto-generated Swagger UI (FastAPI built-in)

7.2 Query Endpoint Contract (as specified in task)

Input:

```
{ "query": "What is the attention mechanism?" }
```

Output:

```
{ "answer": "The attention mechanism allows... [Source: attention.pdf, Page: 3]",  
  "confidence": 0.88, "sources": [{ "source": "attention.pdf", "page": 3, "doc_type":  
  "Technical Document", "score": 0.88 }], "latency_s": 3.42 }
```

7.3 Confidence Score

The confidence score is the cosine similarity of the top retrieved chunk (0.0–1.0). Scores below 0.3 trigger the 'I don't know' fallback without calling the LLM.

8. Key Design Decisions

Decision	Justification
Chunk size 1000 / overlap 500	Suits dense technical PDFs; 50% overlap prevents information loss at chunk boundaries
k=3 retrieved chunks	Balances context richness vs. prompt length; matches reference tutorial approach
Similarity floor 0.3	Prevents LLM from hallucinating on out-of-domain or ambiguous queries
Temperature 0.1	Maximises faithfulness to retrieved context, directly reducing hallucination risk
ChromaDB local store	Zero-infrastructure, disk-persistent, native LangChain integration, no server needed
Gemini 1.5 Flash	State-of-the-art reasoning, 1M token context window, free developer tier
Gemini embeddings	Free, high-quality 768-dim vectors, same API ecosystem as chat model
Batch embed (10 chunks, 15s pause)	Respects Gemini free tier rate limit (100 req/min, 1000 req/day)

9. Folder Structure

Ecell/ pipeline.py .env (GOOGLE_API_KEY) data/ (PDFs) models/chroma/ (vector index) models/eval_results.json requirements.txt README.md
--

10. Limitations & Future Improvements

Limitation	Proposed Improvement
Gemini free tier limits to 1000 embeddings per day	Use local all-MiniLM-L6-v2 model for unlimited local embeddings
Faithfulness metric is keyword-based precision	Use NLI (Natural Language Inference) model for precise groundedness scoring
No reranking step	Add cross-encoder (ms-marco-MiniLM-L-6-v2) for higher precision retrieval
Dense-only retrieval	Add BM25 sparse retrieval + Reciprocal Rank Fusion for better keyword recall
Single-file architecture	Split into src/ modules (preprocess.py, features.py, train.py, evaluate.py, app.py)